Simulador de SO Multitarefa de Tempo Compartilhado

Projeto A — Telas do simulador e explicação de cada elemento visual
CSO30-S73 Sistemas Operacionais · UTFPR Curitiba · Prof. Dr. Marco Aurélio Wehrmeister

Equipe: Marco Gasparotto e Rafael Monçores

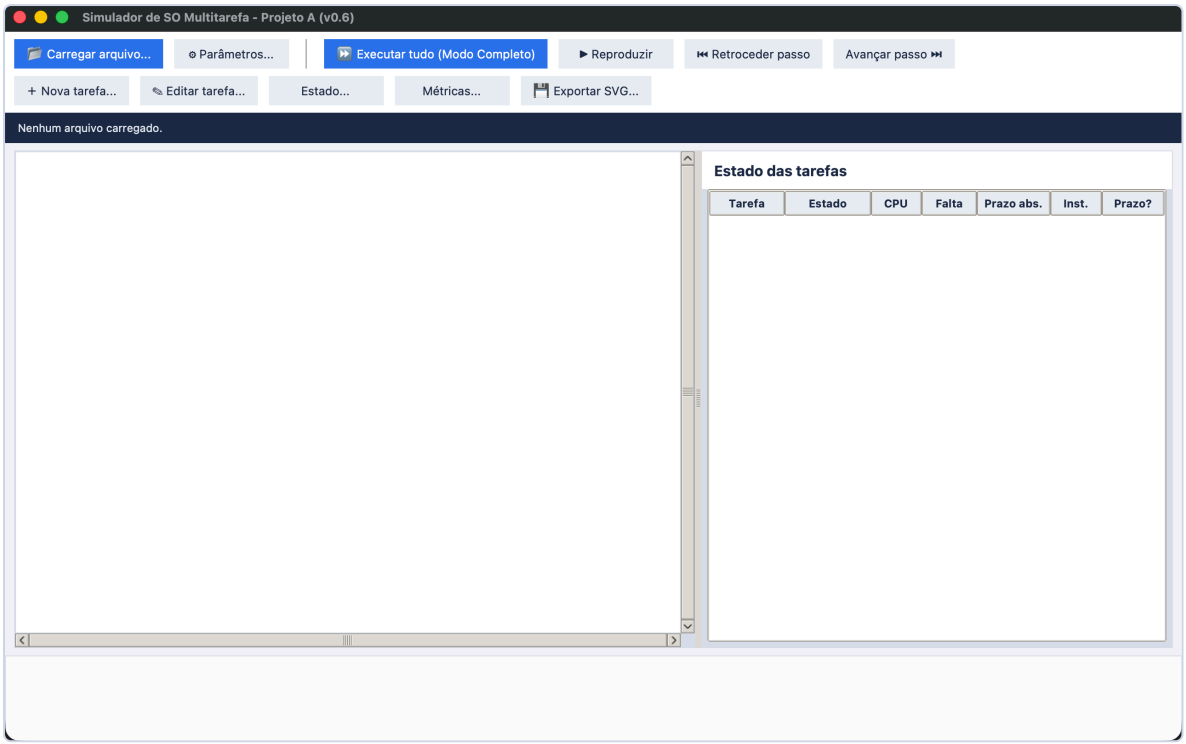
Linguagem: Python 3 com Tkinter (só biblioteca padrão). Execução: `python3 main.py`

Este documento mostra todas as telas criadas no simulador. Para cada tela, explica cada elemento visual e de onde vem a informação mostrada (arquivo de código, classe, função ou campo). As capturas foram feitas com os arquivos de teste da pasta `testes/`.

Organização do código (de onde vêm os dados)

Arquivo	Papel
`configuracao.py`	Lê e valida o arquivo de configuração e gera erros e avisos.
`tarefa.py` / `job.py`	`Tarefa` é a definição estática lida do arquivo. `Job` é cada ativação periódica, com estado, tempo restante, prazo absoluto e métricas.
`simulador.py`	Núcleo: relógio em ticks, CPUs, TCB de cada tarefa, escalonamento, eventos, histórico (snapshots) e métricas.
`escalonadores.py`	Registro plugável de algoritmos (RM e EDF).
`interface.py`	Janela Tkinter: desenha o que o `Simulador` guarda no snapshot do passo exibido.
`exportador_svg.py`	Gera o Gantt completo em SVG.

1. Janela principal (antes de carregar um arquivo)



Esta é a única janela principal. Ela tem cinco áreas, de cima para baixo.

Barra de ferramentas, linha 1: arquivo e controle da simulação

- **Carregar arquivo...** abre o seletor de arquivos (qualquer nome e pasta, req. 3.3.4). O arquivo é lido por `CarregadorConfiguracao.carregar()`, e o `Simulador` é criado e reiniciado.
- **Parâmetros...** abre a tela 7 (algoritmo, quantum e CPUs).
- **Executar tudo (Modo Completo)** roda até o fim sem intervenção (req. 1.5b), via `Simulador.executar_completo()`.
- **Reproduzir / Pausar** avança um passo a cada 350 ms.
- **Retroceder passo** restaura o snapshot anterior, e **Avançar passo** restaura o próximo ou simula um tick novo (`retroceder()` / `avancar()`, req. 1.5.2).

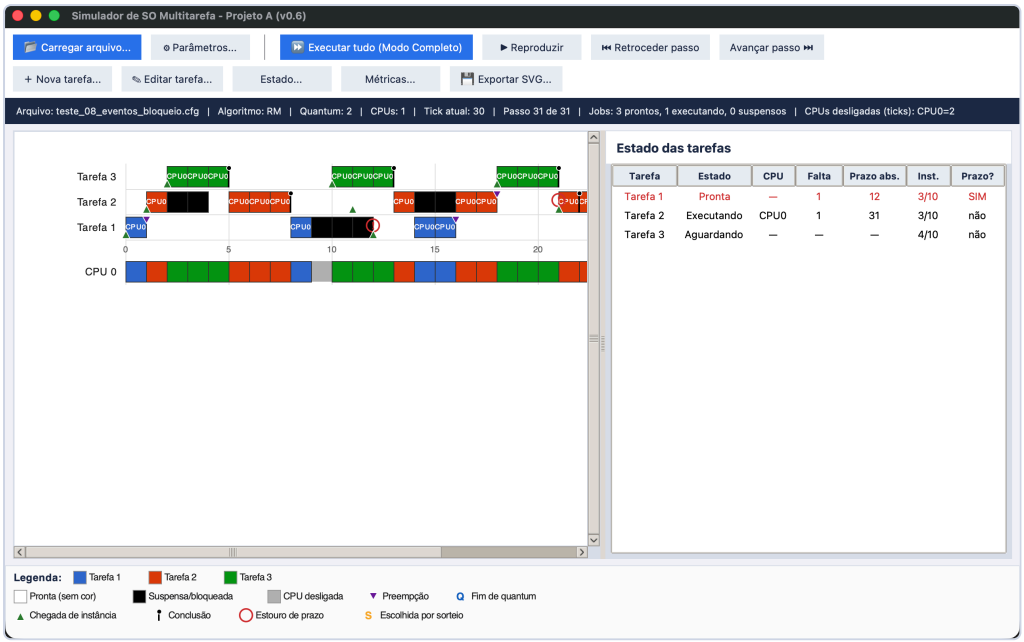
Barra de ferramentas, linha 2: edição e saída

Nova tarefa (tela 8), **Editar tarefa** (tela 9), **Estado** (tela 10), **Métricas** (tela 11) e **Exportar SVG** (salva o Gantt atual em SVG onde o usuário escolher).

Demais áreas

- **Barra de status** (faixa escura): sem arquivo, mostra "Nenhum arquivo carregado". O conteúdo com simulação aparece na tela 2.
- **Área do Gantt** (esquerda) e **painel Estado das tarefas** (direita): ficam vazios até carregar um arquivo.
- **Legenda** (rodapé): é desenhada ao carregar o arquivo.

2. Execução passo a passo: o "debugger" do sistema



Arquivo teste_08_eventos_bloqueio.cfg (RM, 1 CPU, tarefas com eventos de bloqueio), tick 30.

Barra de status (informações do sistema, req. 1.5.1)

Arquivo, algoritmo, quantum e CPUs vêm do `Simulador`. "Tick atual" e "Passo 31 de 31" vêm do snapshot exibido (`historico[indice_historico]`). "Jobs: prontos/executando/suspensos" conta os jobs do snapshot. "CPUs desligadas (ticks)" mostra quantos ticks cada CPU ficou sem tarefa (req. 1.2), via `_ociosidade_por_cpu()`.

Gráfico de Gantt (req. 2.1, 2.2, 2.5)

- **Linhas de tarefa** em ordem decrescente de ID: a Tarefa 1 fica mais perto do eixo X. A **régua** marca os ticks de 5 em 5.
- **Bloco colorido**: a tarefa está executando, na cor do arquivo. O texto **CPU0** no bloco diz em qual processador.
- **Espaço vazio**: a tarefa está pronta na fila, sem executar.
- **Bloco preto** (pontilhado): a tarefa está suspensa por um evento de bloqueio da `lista_eventos` (formato provisório `desloc:dur`).
- **Linha CPU 0**: a cor da tarefa que ocupa a CPU em cada tick. **Cinza** = CPU desligada.
- Os blocos vêm de `Simulador.passo()`, que grava `(tarefa, cpu, início, fim, estado)` por tick no snapshot.

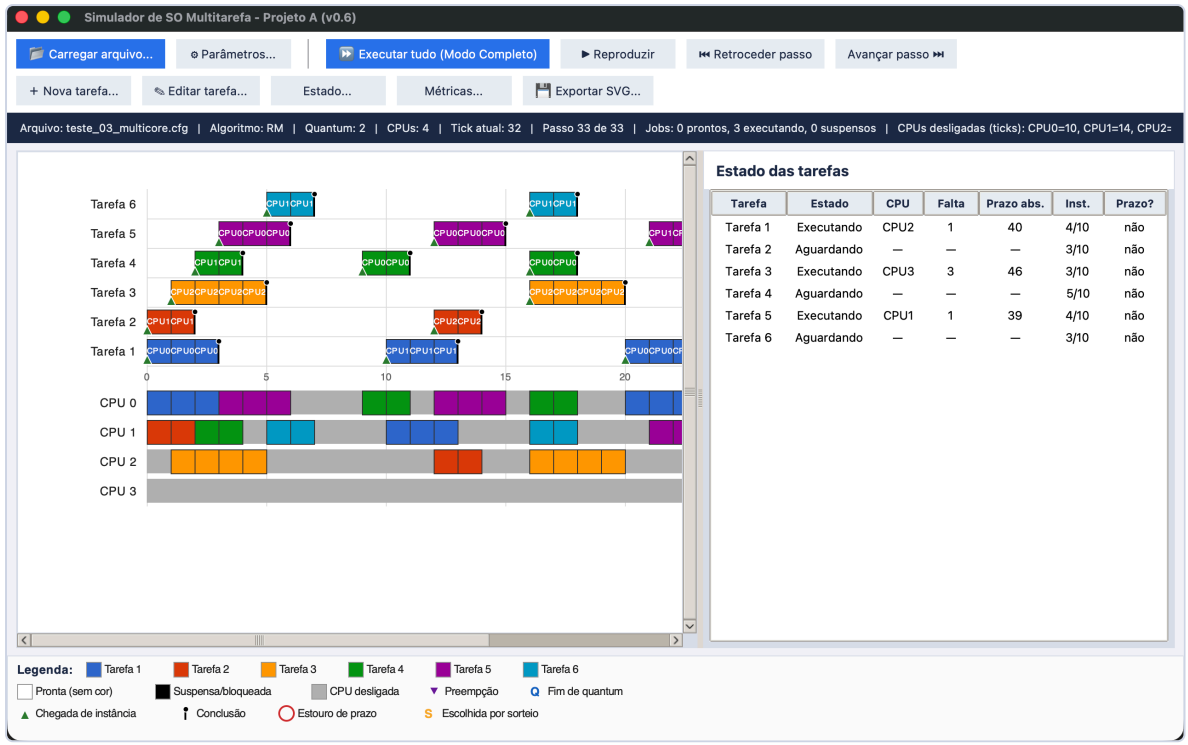
Marcadores de eventos (no tick exato)

▲ chegada de ativação · | ● conclusão · ◯! estouro de prazo · ▼ preempção. Os eventos são gerados em `_processar_tick()` e `_escalonar()`.

Painel Estado das tarefas (TCB, req. 1.3 e 1.5.1)

Uma linha por tarefa, lida do TCB (`estado_tarefas`) do snapshot exibido. Por isso, retroceder também retrocede o painel. **Estado**: Aguardando, Pronta, Executando, Suspensa ou Concluída. **CPU** em uso. **Falta**: ticks restantes da ativação. **Prazo abs.**: liberação + prazo. **Inst.**: ativações liberadas de 10 (req. 4.4). **Prazo?**: SIM se perdeu o prazo; nesse caso a linha fica vermelha.

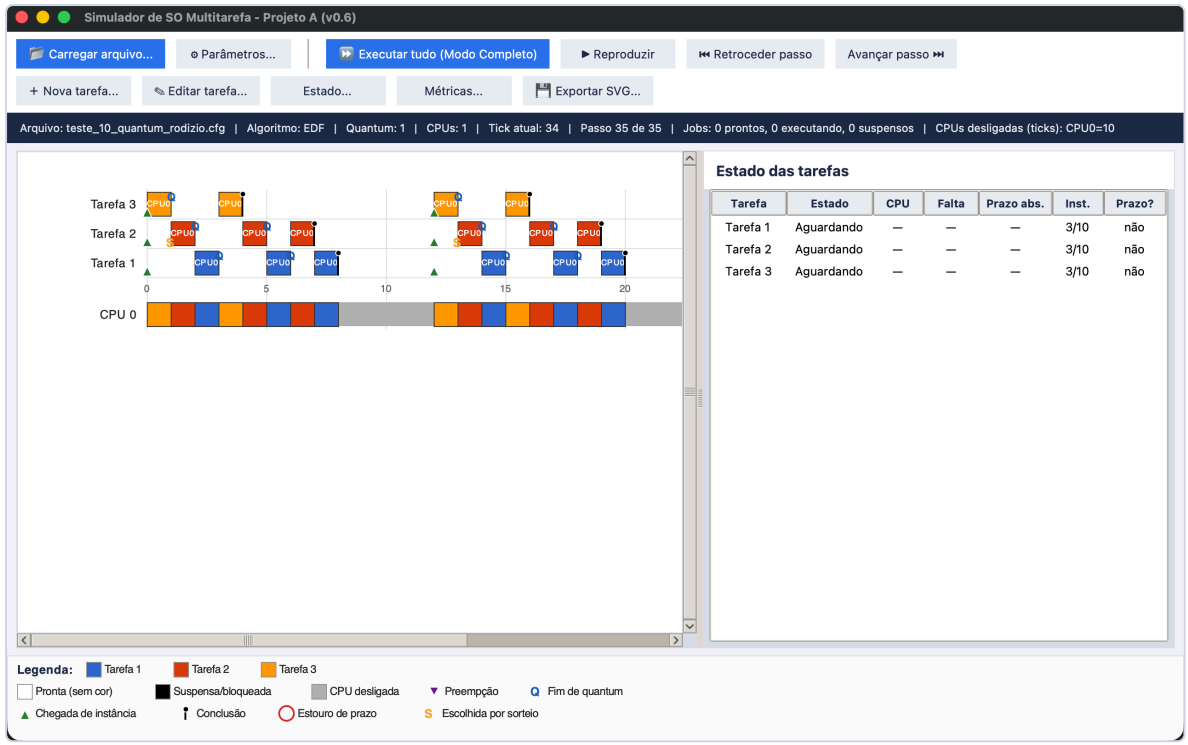
3. Vários processadores e CPU desligada



Arquivo teste_03_multicore.cfg (RM, 4 CPUs, 6 tarefas), tick 32.

- **Uma linha por CPU** (CPU 0 a CPU 3), com a quantidade lida do arquivo (req. geral 2) e alterável em Parâmetros.
- **Fila de prontos global** (req. 1.2): a cada tick, `_escalonar()` ordena todos os jobs elegíveis e entrega as N CPUs aos N primeiros. Nenhuma CPU fica livre se houver tarefa pronta.
- **Cinza** = CPU desligada porque não havia tarefa para ela. A CPU 3 ficou desligada em todo o trecho visível. O tempo desligado de cada CPU aparece na barra de status (`CPU0=10, CPU1=14…`).
- **Texto no bloco** (CPU0, CPU1, CPU2…): processador usado em cada tick. Um job que continua executando mantém a mesma CPU, para evitar migração artificial.
- **Painel**: a coluna CPU mostra onde cada tarefa executa agora. "Aguardando" = já teve ativações e espera a próxima liberação periódica.
- **Legenda**: uma cor por tarefa (`Tarefa.cor`, do arquivo).

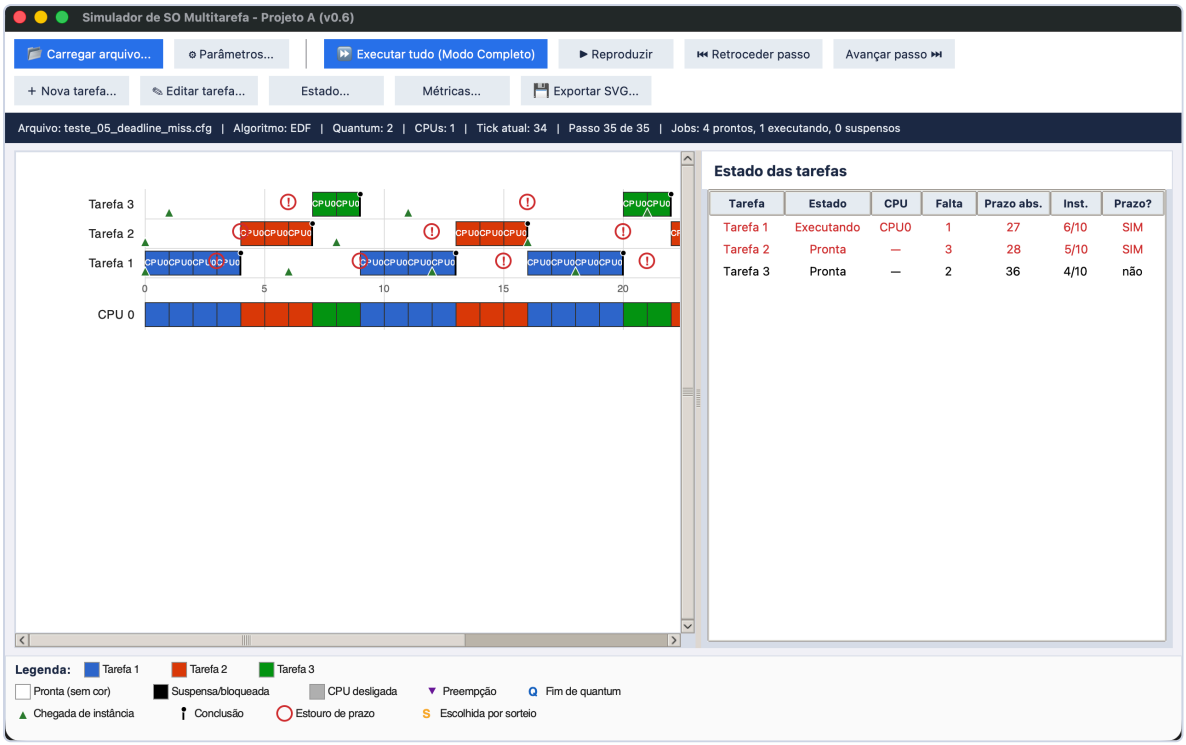
4. Quantum (rodízio) e desempate por sorteio



Arquivo teste_10_quantum_rodizio.cfg (EDF, quantum 1, 3 tarefas com o mesmo prazo).

- **Q azul = fim de quantum.** A tarefa executou `quantum` ticks seguidos e havia outra com a mesma prioridade (mesmo prazo absoluto no EDF, mesmo período no RM). Ela vai para o fim da fila desse nível de prioridade, e as tarefas se revezam (3 → 2 → 1). Se não houver outra de mesma prioridade, ela ganha um quantum novo e continua, sem desrespeitar RM/EDF. Implementado em `Simulador._aplicar_quantum()` e `_chave_job()`.
- **S laranja = escolhida por sorteio** (req. 4.3, critério 5). Aparece quando o empate persiste depois dos critérios 1 a 4 (quem estava executando, prazo, ingresso, duração), no instante em que a tarefa sorteada começa a executar. O sorteio usa um gerador com semente fixa salvo em cada snapshot, então retroceder e avançar repete o mesmo resultado.
- **Linha CPU 0**: a alternância de cores mostra o rodízio. Os trechos cinza são ticks sem nenhuma tarefa pronta.
- **Barra de rolagem horizontal**: o Gantt cresce com o tempo, e a rolagem permite ver toda a linha do tempo.

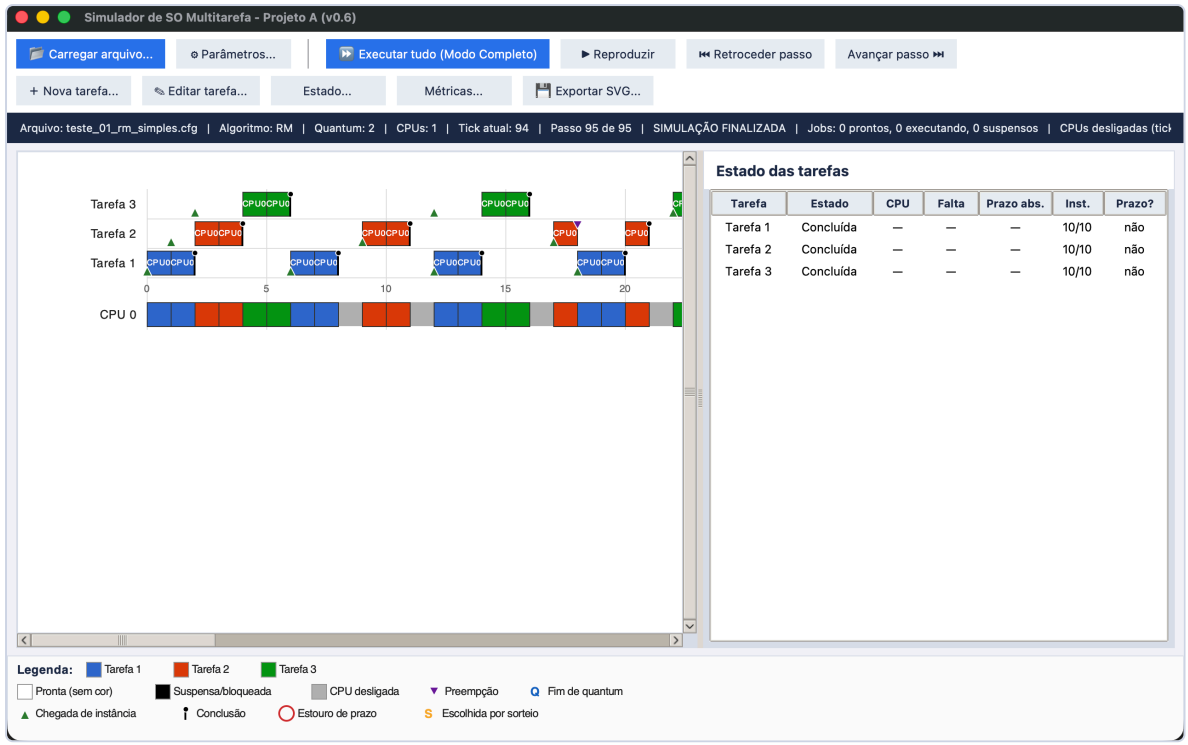
5. Estouro de prazo (deadline miss)



Arquivo teste_05_deadline_miss.cfg (EDF, 1 CPU, sistema sobrecarregado).

- ◯ **! vermelho**: no tick em que o relógio alcança o prazo absoluto de uma ativação ainda não concluída, `_processar_tick()` marca `deadline_perdido` e gera o evento. A conclusão é verificada antes do prazo, então terminar exatamente no prazo não conta como atraso.
- A tarefa que perdeu o prazo **continua executando** até completar sua duração (req. 3.3, campo prazo).
- **Painel**: as linhas das tarefas com prazo perdido ficam vermelhas, e a coluna **Prazo?** mostra SIM.
- O total de prazos perdidos aparece também na tela de Métricas (tela 11).

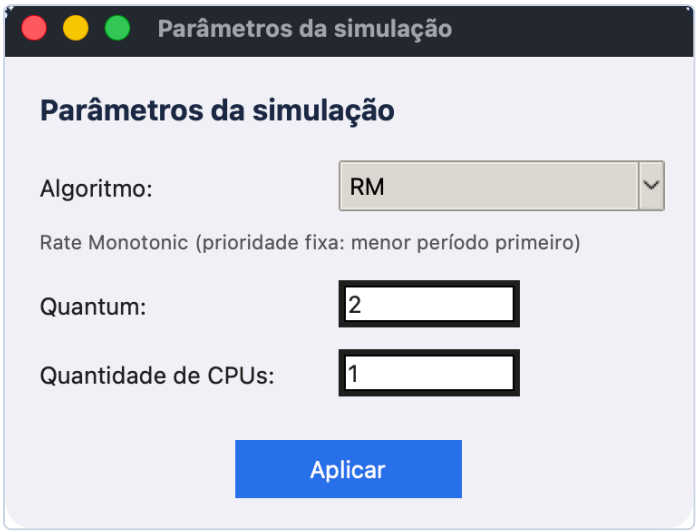
6. Execução completa (sem intervenção humana)



Arquivo teste_01_rm_simples.cfg depois de "Executar tudo".

- "Executar tudo" chama `Simulador.executar_completo()`, que roda até as 10 ativações de cada tarefa terminarem (req. 4.4). Só o resultado final é mostrado (req. 1.5.3).
- **Barra de status**: "SIMULAÇÃO FINALIZADA", tick final e total de passos.
- **Painel**: todas as tarefas em "Concluída" com 10/10 ativações.
- Mesmo depois do fim, os botões Retroceder e Avançar continuam navegando pelo histórico completo.
- Ao terminar, o Gantt é **exportado automaticamente em SVG** ao lado do arquivo carregado (`<nome>_gantt.svg`, req. 2.4). O resultado aparece na tela 15.

7. Parâmetros da simulação



- **Algoritmo**: a lista vem do registro plugável `ALGORITMOS_DE_ESCALONAMENTO` (`escalonadores.py`, req. 4.2). Um algoritmo novo registrado aparece aqui sem alterar a interface.
- **Texto cinza abaixo**: descrição do algoritmo selecionado, também vinda do registro.
- **Quantum** e **Quantidade de CPUs**: vêm preenchidos com os valores atuais (os do arquivo ou os da última edição), como sugere o req. 3.2.
- **Aplicar**: valida os valores (inteiros positivos; caso contrário, mostra uma mensagem de erro com o motivo) e chama `aplicar_edicao_e_reconstruir()`. Ela reinicia e refaz a simulação até o tick em que o usuário estava (req. 3.1: configuração durante a execução).

8. Nova tarefa

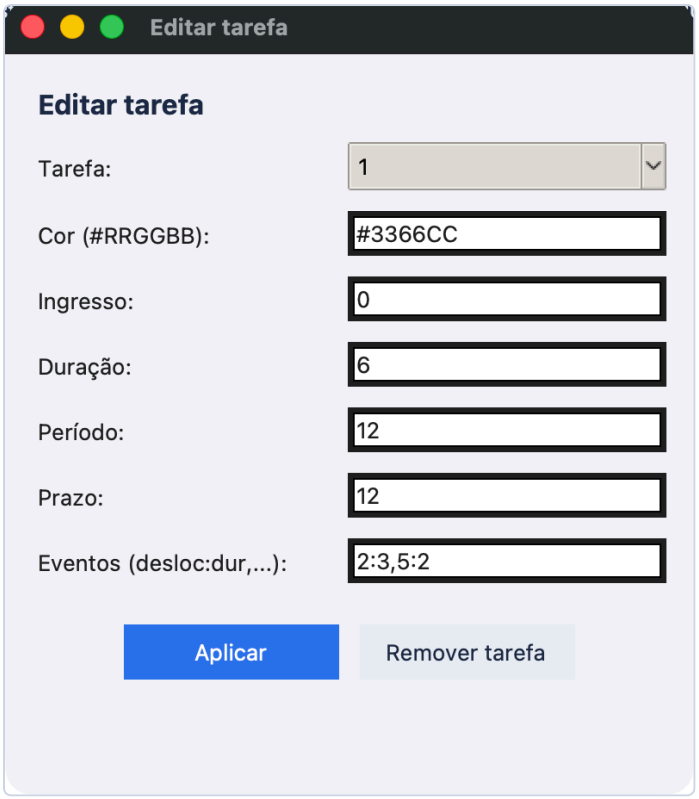


Adiciona uma tarefa ao conjunto simulado durante a simulação (req. 3.1). Todos os campos vêm com **valores padrão sugeridos** (req. 3.2), que o usuário pode sobrescrever:

Campo	Valor sugerido e origem
ID (único)	Maior ID existente + 1
Cor (#RRGGBB)	Primeira cor da `PALETA_CORES_PADRAO` que nenhuma tarefa usa
Ingresso / Duração / Período / Prazo	0 / 1 / 5 / 5
Eventos	Vazio (formato provisório `desloc:dur,...`; a lista de eventos completa é do Projeto B)

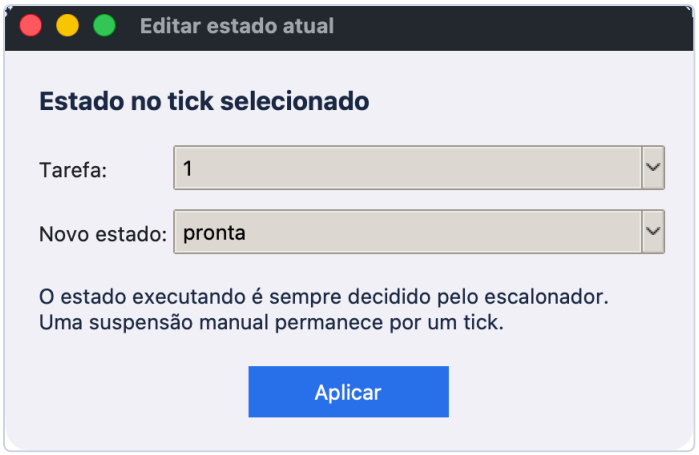
Adicionar tarefa valida tudo. Cada problema gera uma mensagem com o motivo: ID repetido, cor inválida, cor já usada por outra tarefa (cada tarefa precisa de cor diferente, req. 2.1), números fora do intervalo ou evento mal formatado. Depois, a simulação é reconstruída até o tick atual.

9. Editar tarefa



- **Tarefa**: lista os IDs atuais. Ao escolher um ID, os campos são preenchidos com os valores da `Tarefa` correspondente.
- **Cor, Ingresso, Duração, Período, Prazo e Eventos**: as características individuais da tarefa (req. 1.5.2 e 3.1). As validações e mensagens de erro são as mesmas da tela 8.
- **Aplicar**: grava os valores na `Tarefa` e reconstrói a simulação até o tick exibido.
- **Remover tarefa**: pede confirmação e retira a tarefa do conjunto. Não permite remover a última tarefa e explica o motivo.

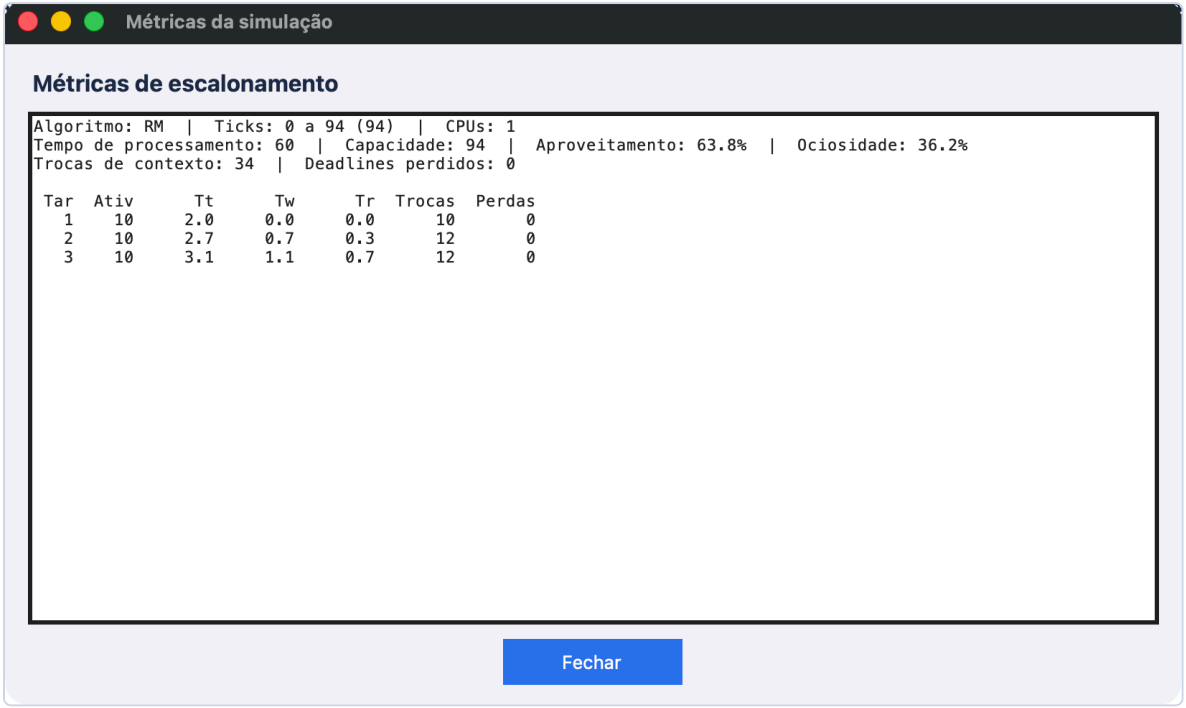
10. Editar estado atual de uma tarefa



Permite modificar manualmente o estado de qualquer tarefa no passo exibido (req. geral 4 e req. 3.4).

- **Tarefa** e **Novo estado**: os estados possíveis são *pronta* e *suspensa*. O texto explicativo informa que "executando" é sempre decidido pelo escalonador e que a suspensão manual dura um tick.
- **Aplicar** chama `Simulador.alterar_estado_tarefa()`. Ela altera o job visível da tarefa, descarta o futuro já simulado (cria uma nova ramificação a partir deste tick) e reescalona.
- **Mudanças inválidas** são recusadas com o motivo. Por exemplo: "A tarefa não possui ativação pendente neste tick."

11. Métricas da simulação



Texto gerado por `Simulador.texto_metricas()` a partir de `calcular_metricas()`, com as métricas do capítulo 6 do livro do Prof. Maziero.

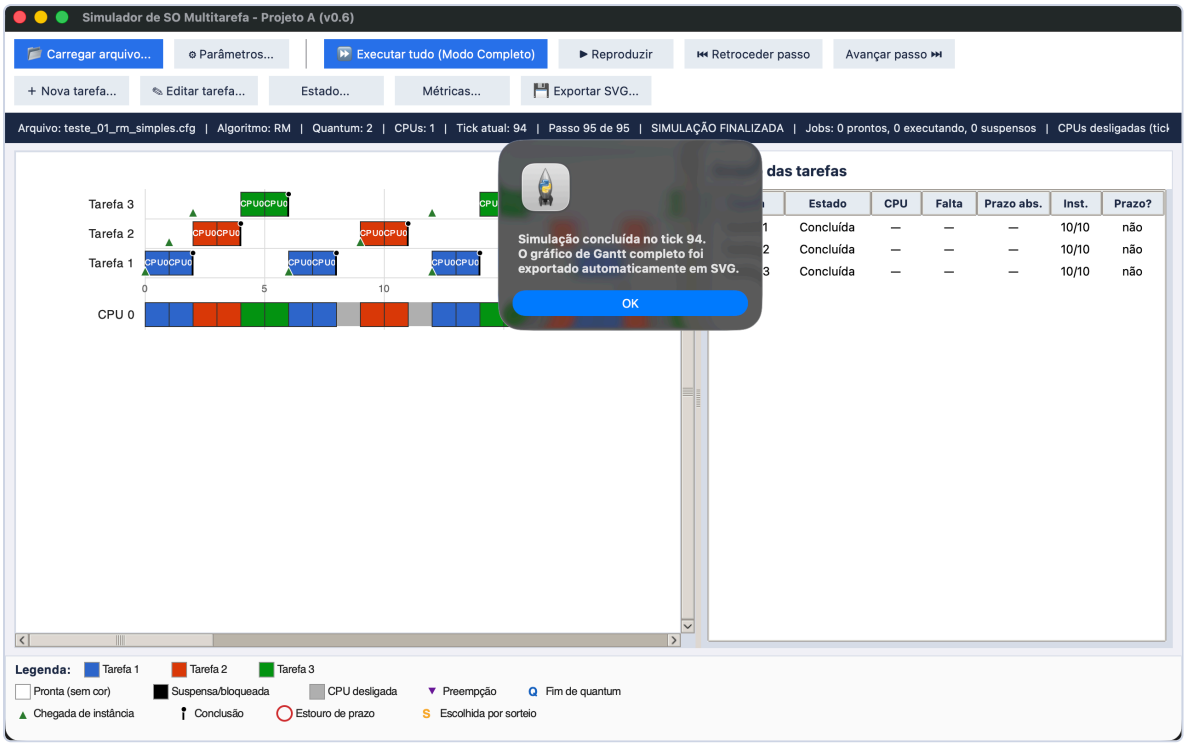
Resumo do sistema

- **Ticks**: do primeiro ao último tick simulado (horizonte).
- **Tempo de processamento**: soma do `tempo_cpu` de todos os jobs. **Capacidade** = CPUs × horizonte.
- **Aproveitamento** = processamento ÷ capacidade. **Ociosidade** = 1 − aproveitamento.
- **Trocas de contexto**: quantas vezes um job entrou numa CPU. **Deadlines perdidos**: jobs com `deadline_perdido`.

Tabela por tarefa (médias das ativações concluídas)

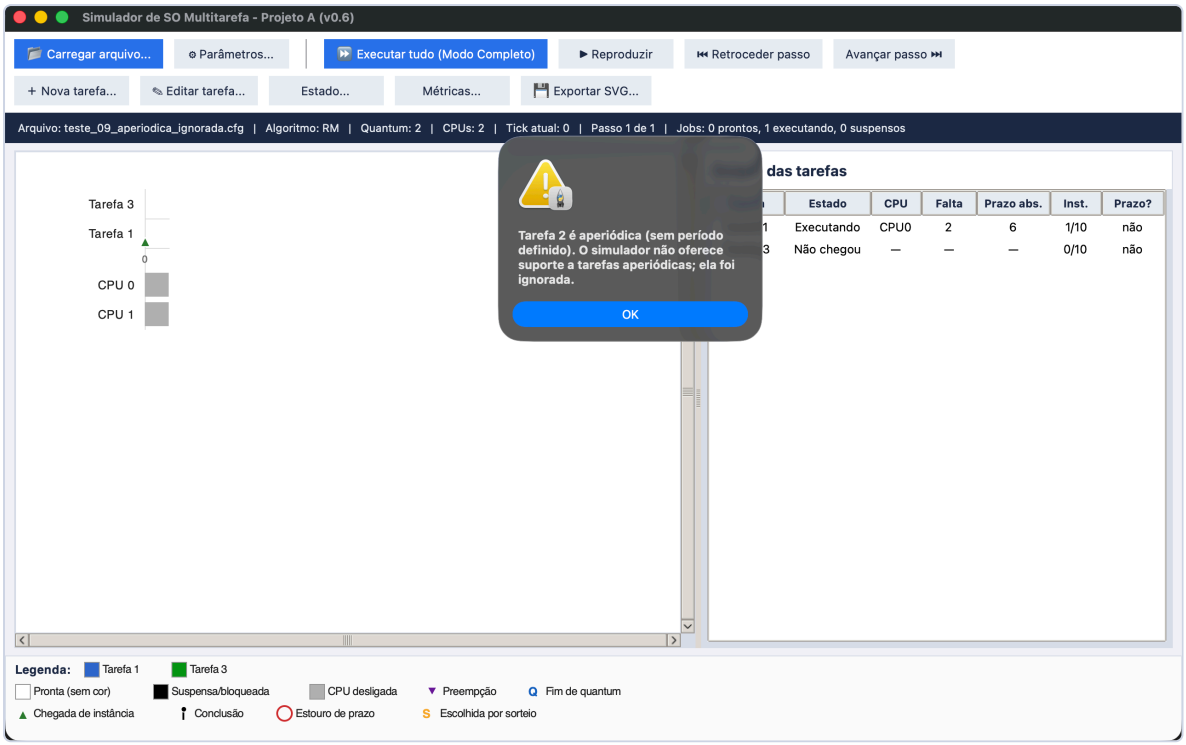
Ativ: ativações concluídas · **Tt**: tempo de vida (conclusão − ingresso) · **Tw**: tempo de espera na fila de prontos · **Tr**: tempo de resposta (primeira execução − ingresso) · **Trocas** · **Perdas**: prazos perdidos.

12. Mensagem de simulação concluída



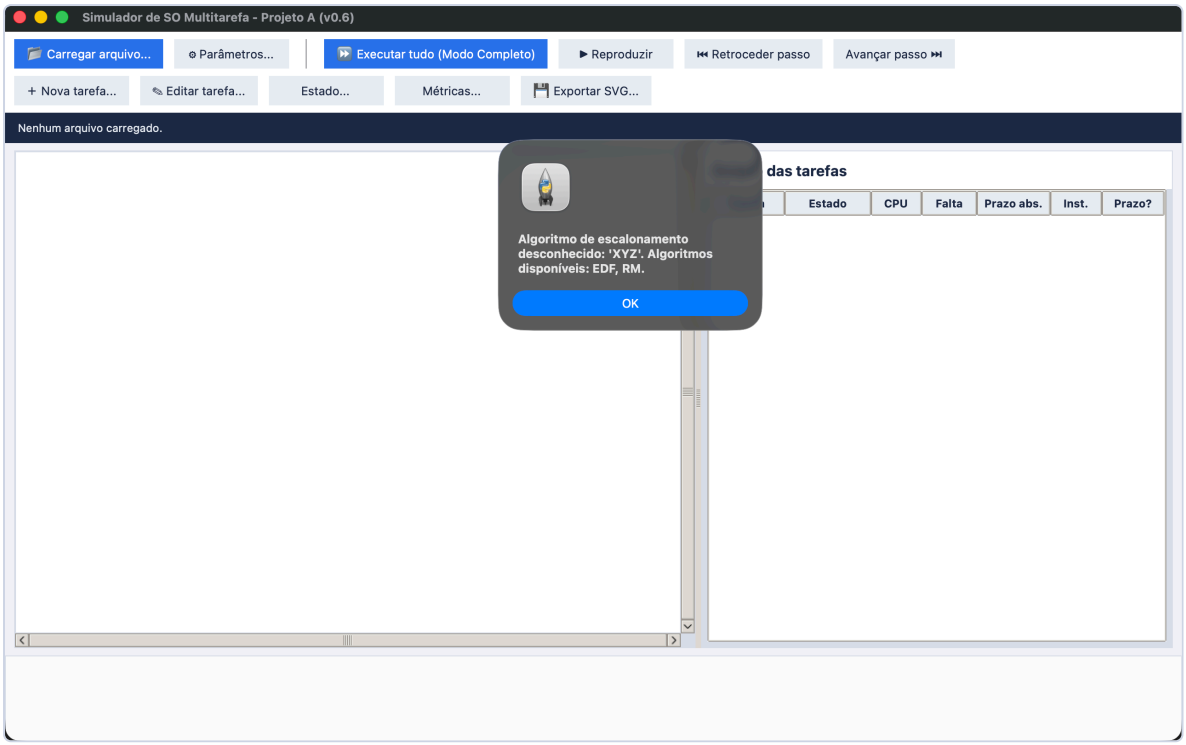
- Aparece ao final do modo completo. Informa o tick em que a simulação terminou (`Simulador.tick`) e que o Gantt completo foi exportado em SVG.
- Atrás da mensagem, a janela já mostra o resultado final: Gantt, status "SIMULAÇÃO FINALIZADA" e painel com as tarefas concluídas.
- O caminho do SVG salvo é acrescentado à barra de status.

13. Aviso ao carregar: tarefa aperiódica ignorada



- No Projeto A, só tarefas periódicas são simuladas. Uma tarefa com período 0 (ou vazio) é **ignorada e o usuário é avisado** (req. 4.4). O arquivo continua carregando normalmente: o Gantt e o painel ao fundo mostram só as tarefas 1 e 3.
- Outros avisos que usam esta mesma janela:
  - Evento fora do formato provisório: os eventos daquela tarefa são ignorados.
  - Cor repetida no arquivo: a tarefa repetida recebe uma cor livre da paleta, para o Gantt continuar legível.
- Os avisos são acumulados por `CarregadorConfiguracao` em `Configuracao.avisos`.

14. Erro ao carregar o arquivo de configuração



Erros no arquivo são mostrados com o **motivo exato** (req. 3.3.6 e 5), e a simulação anterior não é alterada. No exemplo, o algoritmo "XYZ" não existe, e a mensagem lista os algoritmos disponíveis.

Outras mensagens produzidas por `CarregadorConfiguracao`:

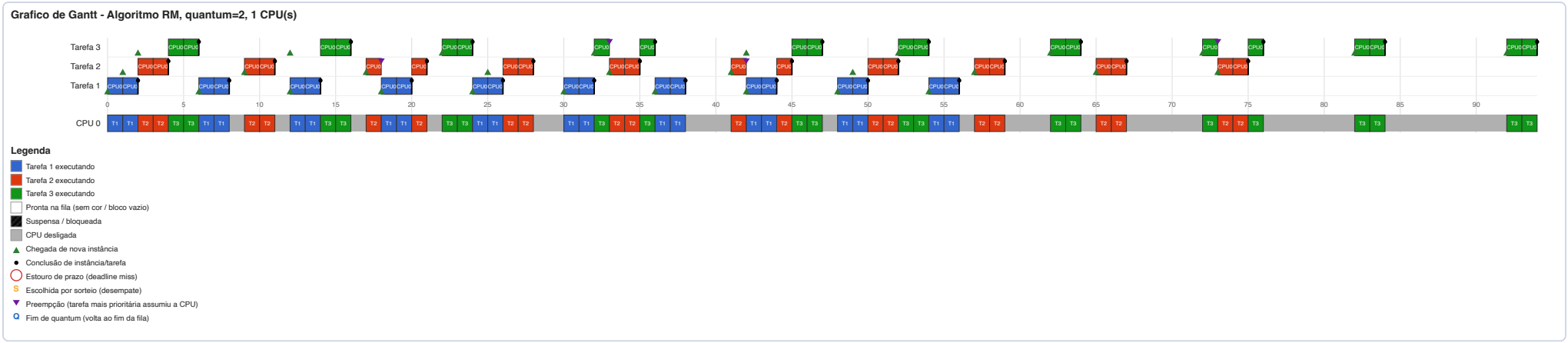
- "Arquivo não encontrado"; "O arquivo de configuração está vazio".
- "Primeira linha inválida", com o formato esperado; "Quantum inválido"; "Quantidade de CPUs inválida".
- "Linha N: esperados 7 campos"; "ID inválido/duplicado"; "cor inválida"; "ingresso/duração/período/prazo inválido".

Não são erros (req. 3.3.2, 3.3.3, 3.3.6): maiúsculas e minúsculas misturadas, espaços, linhas em branco, ";" no fim da linha e BOM no início do arquivo.

15. Imagem final exportada (SVG)

Gerada por `ExportadorSVG.exportar()` só com concatenação de texto, sem bibliotecas externas (req. 2.4). Não é um print da tela: contém a simulação inteira, do primeiro ao último tick, sem limite de tempo. A largura depende só da duração da simulação. O título traz o algoritmo, o quantum e o número de CPUs. Os elementos e a legenda são os mesmos do Gantt da tela (ID menor mais perto do eixo, blocos por CPU, cinza = CPU desligada, preto hachurado = suspensa, e marcadores de chegada, conclusão, prazo perdido, sorteio, preempção e fim de quantum).

Imagem completa (teste_01_rm_simples.cfg, 94 ticks)



Ampliação dos primeiros ticks

